FREE COURSE · 14 MODULES + LABS

# AI Engineering, Assembled in Order

*A build-log course from first principles to production — each module is a component wired to the one before it.*

THE BUILD, MODULE 01 → 14

Edition 01
Prepared October 2026

AI ENGINEERING /14

Read in order — the trace rail at the top of every page marks your position in the build.

# Fourteen Modules, One Pipeline

Each module builds on the last, and each ends with a hands-on lab that extends one running project. The rail at the top of every page marks your position — read in order the first time through.


| | | |
|---|---|---|
| 01 | **Introduction to AI Engineering** | *Why this discipline exists, and where it sits between ML research and software engineering* |
| 02 | **LLMs** | *How large language models actually work, at the depth an engineer needs* |
| 03 | **Prompt Engineering** | *Directing model behaviour reliably through instructions, structure, and examples* |
| 04 | **OpenAI API** | *Talking to a hosted LLM programmatically: requests, parameters, streaming, cost* |
| 05 | **Embeddings** | *Turning meaning into coordinates — the substrate underneath search and RAG* |
| 06 | **Vector Databases** | *Storing and searching millions of embeddings fast* |
| 07 | **RAG** | *Grounding generation in real, current, retrievable knowledge* |
| 08 | **Function Calling** | *Letting the model reach for code and APIs instead of only text* |
| 09 | **MCP** | *A standard wire protocol between models and the tools/data they use* |
| 10 | **AI Agents** | *Loops, not single calls: planning, acting, observing, and deciding what's next* |
| 11 | **LangGraph** | *Building controllable, stateful multi-step agent workflows as a graph* |
| 12 | **Evaluation** | *Knowing whether any of this actually works, with numbers instead of vibes* |
| 13 | **Production AI** | *Latency, cost, safety, and failure handling once real users show up* |
| 14 | **MLOps** | *Keeping models and pipelines reliable, versioned, and observable over time* |

# Introduction to AI Engineering

> *Why this discipline exists, and the system you'll assemble module by module.*

## WHAT AI ENGINEERING IS

AI engineering is the discipline of building products on top of pre-trained foundation models. Unlike classical machine learning, you rarely train a model yourself — you compose systems around models that already exist: prompting them, grounding them in your data, giving them tools, and wrapping them in evaluation and operations so they behave reliably for real users.

That shift changes the skill set. The centre of gravity moves from datasets and training loops to **system design**: context construction, retrieval, orchestration, guardrails, and measurement. Most of this course is about exactly those layers.




FIGURE 1.1 - THE COURSE IN ONE LINE: EACH LAYER WRAPS THE PREVIOUS ONE

## HOW IT DIFFERS FROM ML ENGINEERING

- **Model:** consumed via API or open weights, not trained from scratch.
- **Data work:** curating context and knowledge bases, not labelled training sets.
- **Iteration loop:** edit a prompt or pipeline and re-run in seconds, not retrain for days.
- **Failure modes:** hallucination, prompt injection, drift — not just accuracy loss.

MODULE 01 — KEY TAKEAWAYS

- You build *systems around models*, not models themselves.
- The stack you will assemble: prompting → retrieval → tools → agents → evaluation → operations.
- Fast iteration is the superpower — design your workflow so a change can be tested in minutes.

## LAB 01 — EXERCISES

No code yet — this lab sets up the project and the habits the rest of the course builds on.

### E1 · Map a real product [warm-up]

Pick an AI product you use (a coding assistant, a support bot, a search feature). Identify which layers of the stack from Figure 1.1 it uses, and which it visibly lacks.

### E2 · Failure taxonomy [core]

List five realistic ways that product could fail. Classify each as hallucination, retrieval miss, prompt injection, latency/cost, or tooling error. Note which layer would catch it.

### E3 · Design your iteration loop [core]

Write down the exact steps to test a prompt change in your own setup. If the loop takes more than five minutes, redesign it until it doesn't — this loop is your main asset for the rest of the course.

SHIP IT — LAB CHALLENGE

Write a one-page build spec for a small AI product you will construct across these labs (examples: a docs Q&A bot, a support triager, a research summarizer). State the user, the task, the data it needs, and which modules of this course map to which part. Every later lab extends this project.

▸ Reference code: **labs/01-intro** in the companion repo. Try the exercises first, then compare.

# LLMs

> *How next-token prediction becomes conversation — and what that implies for engineering.*

## NEXT-TOKEN PREDICTION, SCALED UP

A large language model is a **transformer** trained on one deceptively simple objective: given the tokens so far, predict the next one. Trained across trillions of tokens, that objective forces the network to internalise grammar, facts, style, and a workable approximation of reasoning — because all of those help predict what comes next.

Generation is the same trick run in a loop: the model outputs a probability distribution over its vocabulary, one token is sampled from it, appended to the input, and the process repeats until a stop condition.




FIGURE 2.1 - ONE GENERATION STEP; THE SAMPLED TOKEN LOOPS BACK AS INPUT

## THE NUMBERS THAT MATTER

- **Tokens** — subword chunks (≈4 characters of English each). You are billed and limited in tokens, not words.
- **Context window** — the maximum tokens the model can attend to at once (input + output). Everything you want it to know *right now* must fit here.
- **Temperature** — sampling randomness. Low (≈0) for deterministic extraction; higher (≈0.7–1.0) for creative variety. Some newer models fix or ignore this setting.
- **Training cutoff** — the model knows nothing after this date without retrieval.

## CAPABILITY VS. RELIABILITY

LLMs are probabilistic. The same prompt can produce different outputs; confident prose can be factually wrong (*hallucination*). Treat raw model output the way you treat user input: validate it, constrain it, and never assume correctness. Most of the rest of this course — retrieval, tools, evaluation — exists to convert raw capability into engineering reliability.

MODULE 02 — KEY TAKEAWAYS

- An LLM predicts tokens; everything else is emergent behaviour on top of that.
- Context window is working memory — finite, priced, and precious.
- Design for probabilistic output: validate, constrain, verify.

## LAB 02 — EXERCISES

Goal: build physical intuition for tokens, sampling, and context before writing any product code.

### E1 · Count tokens [warm-up]

Use `tiktoken` to count tokens for three texts: a tweet, a page of prose, and a code file. Predict the API cost of each at your model's current per-token price.

### E2 · Temperature sweep [core]

Send the same prompt at temperature 0, 0.7, and 1.2, three samples each. Record the variance. Decide which setting your Lab-01 project needs and why.

### E3 · Context-window stress test [stretch]

Hide one fact ("the access code is 7431") inside progressively longer padding documents and ask the model to recall it. Chart recall vs. context length for your model.

SHIP IT — LAB CHALLENGE

Write a half-page *model card summary* for the model your project will use: context window, training cutoff, per-token cost in and out, and the temperature you chose, with one sentence of justification each.

▸ Reference code: **labs/02-llms** in the companion repo. Try the exercises first, then compare.

# Prompt Engineering

> *The cheapest, fastest lever for controlling model behaviour.*

## THE CHEAPEST LEVER YOU HAVE

Prompt engineering is directing model behaviour through the input itself. It is the first tool to reach for because it costs nothing to change and seconds to test. A well-structured prompt routinely outperforms a mediocre fine-tune.

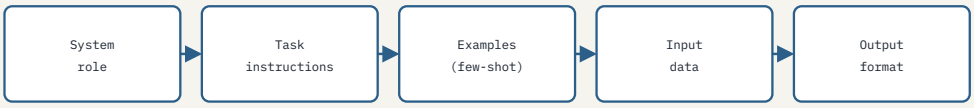



FIGURE 3.1 - ANATOMY OF A PRODUCTION PROMPT, IN THE ORDER THE MODEL READS IT

## TECHNIQUES THAT ACTUALLY MOVE THE NEEDLE

- **Be explicit and specific.** State the task, audience, constraints, and what to do in edge cases. Ambiguity in, variance out.
- **Few-shot examples.** Two or three input→output pairs teach format and tone faster than paragraphs of description.
- **Chain of thought.** Asking the model to reason step-by-step before answering measurably improves multi-step tasks.
- **Structured output.** Demand JSON with a stated schema (or use the API's structured-output mode) so downstream code can parse the result.
- **Delimiters.** Fence user data in tags like `<input>...</input>` so instructions and data cannot blur together.

## PROMPTS ARE CODE

Keep prompts in version control, not in a dashboard textbox. Write down expected outputs for a handful of test inputs, and re-run them after every prompt edit — this is the seed of the evaluation habit Module 12 formalises. And never concatenate untrusted user text straight into instructions: that is how **prompt injection** happens.

MODULE 03 — KEY TAKEAWAYS

- Structure beats cleverness: role, instructions, examples, data, format.
- Version prompts and test them like code.
- Separate instructions from untrusted data — always.

## LAB 03 — EXERCISES

Everything here uses the API you'll formalise in Module 04 — the repo provides a thin helper so you can focus on the prompts.

### E1 · Rewrite a vague prompt [warm-up]

Take “summarize this” and rebuild it with the Figure 3.1 anatomy: role, explicit instructions, edge-case behaviour, and a required output format.

### E2 · Few-shot format lock [core]

Add three input→output examples to a JSON-extraction prompt. Run it ten times with and without them; measure how often the output parses cleanly in each condition.

### E3 · Break in, then lock out [core]

Craft a user input that hijacks your own prompt (“ignore previous instructions…”). Then defend: fence user data in delimiters and re-test the same attack.

SHIP IT — LAB CHALLENGE

Commit your project's prompts as versioned files ( `prompts/v1_….md` ) plus a five-case mini-eval script that runs them and checks outputs. This becomes the seed of Lab 12.

▸ Reference code: **labs/03-prompt-engineering** in the companion repo. Try the exercises first, then compare.

# OpenAI API

> *Requests, streaming, retries, and cost: talking to a hosted model like an engineer.*

## THE SHAPE OF EVERY CALL

Nearly every hosted LLM — OpenAI, Anthropic, and most others — exposes the same core pattern: send an HTTPS request containing a **list of messages** (each with a role: `system`, `user`, or `assistant`), receive the model's reply as a new message. The API is **stateless**: the model remembers nothing between calls, so your application resends whatever history matters every time.

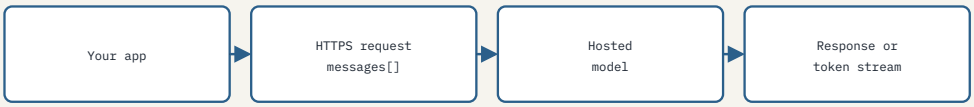



FIGURE 4.1 - STATELESS REQUEST/RESPONSE; MEMORY LIVES IN YOUR CODE, NOT THE MODEL

## A MINIMAL CALL

```python
# pip install openai
import os
from openai import OpenAI
client = OpenAI()  # reads OPENAI_API_KEY from env

resp = client.chat.completions.create(
    model=os.environ["CHAT_MODEL"],  # a current model id, kept in config
    messages=[
        {"role": "system", "content": "You are a concise assistant."},
        {"role": "user",   "content": "Explain embeddings in one line."},
    ],
    temperature=0.2,
)
print(resp.choices[0].message.content)
```

## PRODUCTION CONCERNS FROM DAY ONE

- **Streaming** — request token-by-token output so users see text immediately instead of waiting for the full response.
- **Errors & retries** — handle rate limits (HTTP 429) and transient failures with exponential backoff.
- **Cost** — you pay per input + output token; log `usage` from every response and watch prompt bloat.
- **Config, not constants** — providers retire models and change prices. Keep model IDs and rates in configuration.
- **Secrets** — API keys live in environment variables or a secret manager, never in code or a browser.

MODULE 04 — KEY TAKEAWAYS

- Messages in, message out, no server-side memory — you own the conversation state.
- Stream for UX, retry with backoff, log token usage from the first prototype.

## LAB 04 — EXERCISES

This lab produces the API wrapper the rest of the course runs on — invest here.

### E1 · Minimal call [warm-up]

Reproduce the module's minimal call, then refactor it into a reusable `ask(messages)` helper your later labs will import.

### E2 · Streaming [core]

Switch the helper to streaming and print tokens as they arrive. Measure time-to-first-token vs. time-to-full-response for the same request.

### E3 · Retries with backoff [core]

Wrap the call in exponential backoff for HTTP 429/5xx (the repo's chaos test shows how to simulate failures). Verify it survives three consecutive simulated failures.

### E4 · Cost ledger [stretch]

Log model, input tokens, output tokens, and computed cost for every call to a CSV. Keep model IDs and prices in configuration, not in code.

SHIP IT — LAB CHALLENGE

Build a tiny CLI chatbot: keeps conversation history, streams responses, retries on failure, and prints the running cost after each turn. This wrapper is the engine for every later lab.

▸ Reference code: **labs/04-openai-api** in the companion repo. Try the exercises first, then compare.

# Embeddings

> *Meaning as coordinates: the substrate under semantic search and RAG.*

## MEANING AS COORDINATES

An **embedding** is a fixed-length vector of numbers — typically a few hundred to a few thousand dimensions — produced by a model trained so that semantically similar text lands close together in that space. “Reset my password” and “I can't log in” share almost no words, but their embeddings sit near each other; that is what makes semantic search possible where keyword search fails.

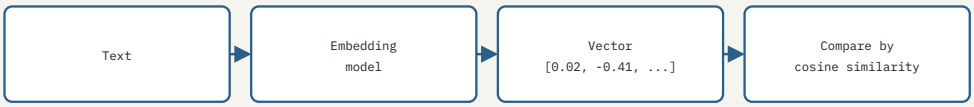



FIGURE 5.1 - TEXT IN, COORDINATES OUT; SIMILARITY BECOMES GEOMETRY

## WORKING WITH EMBEDDINGS

- **Similarity metric.** Cosine similarity is the default: 1.0 means same direction (same meaning), near 0 means unrelated.
- **Chunking.** Long documents are split into passages (commonly a few hundred tokens, often with overlap) and each chunk is embedded separately — retrieval quality depends heavily on chunk boundaries.
- **One model per index.** Vectors from different embedding models are incompatible; changing models means re-embedding the corpus.
- **Beyond search:** clustering, deduplication, recommendation, and classification all fall out of “meaning as distance.”

## WHERE THIS SLOTS INTO THE BUILD

Embeddings are infrastructure, not a product feature by themselves. In Module 06 we give them a home that can search millions of vectors in milliseconds; in Module 07 they become the retrieval half of RAG.

MODULE 05 — KEY TAKEAWAYS

- Embeddings turn “how similar is this?” into a distance computation.
- Chunking strategy quietly decides your retrieval quality — treat it as a design choice.
- Never mix vectors from different embedding models in one index.

## LAB 05 — EXERCISES

Everything is small enough to run with in-memory arrays — you'll feel exactly what a vector database automates in Module 06.

### E1 · Similarity ranking [warm-up]

Embed ten sentences (mix two topics). Compute the cosine-similarity matrix and confirm the two topic clusters are visible in the numbers.

### E2 · Chunkers, compared [core]

Implement fixed-size and paragraph-aware chunking over the same document. For five test questions, check which chunker's top-3 chunks actually contain the answer.

### E3 · Near-duplicate detector [stretch]

Embed a folder of notes and flag pairs above 0.92 similarity. Tune the threshold until false positives disappear.

SHIP IT — LAB CHALLENGE

Build *semantic grep*: a CLI that takes a query and a folder, embeds both, and prints the five most relevant passages with scores. No vector DB yet — plain arrays are the point.

▸ Reference code: **labs/05-embeddings** in the companion repo. Try the exercises first, then compare.

# Vector Databases

> *Approximate nearest neighbours: searching millions of vectors in milliseconds.*

## THE PROBLEM: NEAREST NEIGHBOURS AT SCALE

Comparing a query vector against ten thousand stored vectors is a simple loop. Against fifty million, brute force is too slow. A **vector database** solves this with *approximate nearest neighbour* (ANN) indexes — structures like HNSW graphs that find the closest vectors in milliseconds by accepting a tiny, tunable chance of missing the exact best match.

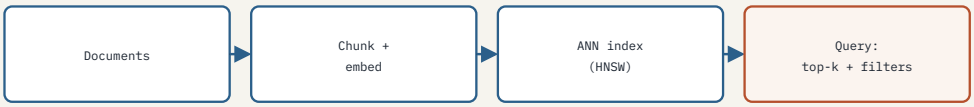



FIGURE 6.1 - THE INGESTION AND QUERY PATH EVERY VECTOR STORE SHARES

## WHAT YOU ACTUALLY STORE

Each record is a triple: the **vector**, the original **text chunk**, and **metadata** (source, date, tenant, permissions). Metadata matters more than it looks: real queries are "top-5 most similar *where product = X and customer = Y*", and filtered search is where engines differ most.

## CHOOSING ONE

- **pgvector** — an extension to PostgreSQL; the right default when your data already lives there and scale is moderate.
- **Dedicated engines** (Pinecone, Weaviate, Qdrant, Milvus) — managed scaling, hybrid keyword+vector search, and tenant isolation for larger corpora.
- **Libraries** (FAISS, Chroma) — in-process indexes, ideal for prototypes and evaluation harnesses.

Whichever you pick, plan for **re-indexing**: content updates, chunking changes, and embedding-model upgrades all require rebuilding vectors. Treat the index as a derived artifact of your source data, never the source of truth.

MODULE 06 — KEY TAKEAWAYS

- ANN indexes trade a sliver of recall for orders-of-magnitude speed.
- Store text + metadata with every vector; filtered search is the real workload.
- The index is derived data — keep a path to rebuild it from source.

## LAB 06 — EXERCISES

The lab treats the index as derived data from day one — the rebuild script is the real deliverable.

### E1 · First index `[warm-up]`

Load the sample docs into Chroma with source and date metadata. Query top-5 for three questions and eyeball the hits.

### E2 · Filtered search `[core]`

Repeat the queries with a metadata filter ( `source = …` ). Find one query whose answer changes — that's the filtered-search workload in action.

### E3 · Recall check `[stretch]`

At this small scale you can brute-force exact nearest neighbours. Compare against the index's results and compute recall@5.

SHIP IT — LAB CHALLENGE

Write an idempotent `reindex.py` : wipe and rebuild the collection from source files, stamping each record with an embedding-model name and content hash. Re-running it twice must produce the identical index.

▸ Reference code: **labs/06-vector-db** in the companion repo. Try the exercises first, then compare.

# RAG

> *Grounding generation in your data — retrieval and prompt discipline combined.*

## CLOSING THE KNOWLEDGE GAP

**Retrieval-Augmented Generation** connects the two halves you just built. The model's weights are frozen at its training cutoff and know nothing about your private data. RAG fixes both limits at answer time: retrieve the most relevant chunks for the user's question, place them in the prompt, and instruct the model to answer *from that context*.




FIGURE 7.1 - THE RAG PIPELINE: MODULES 05 AND 06 FEEDING MODULE 02

## THE PROMPT SIDE OF RAG

Retrieval is only half the system. The generation prompt should: fence the retrieved chunks in clear delimiters, tell the model to answer **only** from the provided context, require it to say "I don't know" when the context doesn't contain the answer, and ask for citations back to chunk identifiers so users can verify claims.

## WHY RAG QUALITY IS RETRIEVAL QUALITY

When a RAG system answers badly, the cause is usually upstream of the model: the right chunk wasn't retrieved, was split awkwardly, or was buried under near-duplicates. The levers that matter, roughly in order:

- **Chunking** — respect document structure (headings, paragraphs) rather than fixed character counts.
- **Hybrid search** — combine vector similarity with keyword matching; exact terms like error codes and product names need lexical search.
- **Reranking** — retrieve 20–50 candidates cheaply, then use a stronger reranker model to pick the best 3–5 for the prompt.
- **Freshness** — re-embed content when it changes; stale indexes produce confidently outdated answers.

MODULE 07 — KEY TAKEAWAYS

- RAG = search + prompt discipline; the model is the last step, not the system.
- Instruct the model to refuse when context is insufficient — that is a feature.
- Debug retrieval first: most "hallucinations" are retrieval misses.

## LAB 07 — EXERCISES

This is the payoff lab — three modules of infrastructure become one working system.

### E1 · Wire it together [warm-up]

Connect Lab 05's chunks, Lab 06's index, and Lab 04's wrapper into one `answer(question)` function that cites chunk IDs.

### E2 · Honest refusals [core]

Add the “answer only from context; say you don't know otherwise” instruction. Test with three questions your corpus cannot answer and verify it refuses all three.

### E3 · Hybrid retrieval [core]

Add a keyword pre-filter for exact terms (error codes, product names). Find one query where hybrid beats pure vector search.

### E4 · Rerank [stretch]

Retrieve 20 candidates, rerank to 5 with a stronger model, and compare answer quality on your test questions.

SHIP IT — LAB CHALLENGE

Point the pipeline at this course's own PDF: chunk it, index it, and answer five questions about the course with correct module citations. If it can't cite Module 07 explaining RAG, debug retrieval first.

▸ Reference code: **labs/07-rag** in the companion repo. Try the exercises first, then compare.

# Function Calling

> *The model proposes structured calls; your code executes them.*

## FROM TEXT TO ACTION

**Function calling** lets a model do more than write prose. You describe the functions your application exposes — name, purpose, JSON-schema parameters — and the model, when it decides a function would help, replies not with text but with a structured call: *which* function and *what* arguments. Crucially, **the model never executes anything**. Your code runs the function, sends the result back, and the model composes the final answer.




FIGURE 8.1 - THE TWO-ROUND-TRIP PATTERN BEHIND EVERY TOOL-USING ASSISTANT

## DECLARING A TOOL

```
tools = [{
  "type": "function",
  "function": {
    "name": "get_order_status",
    "description": "Look up the shipping status of an order.",
    "parameters": {
      "type": "object",
      "properties": {"order_id": {"type": "string"}},
      "required": ["order_id"]
    }
  }
}]
```

## ENGINEERING THE BOUNDARY

- **Descriptions are prompts.** The model chooses tools by reading your descriptions — write them for the model, with when-to-use guidance.
- **Validate everything.** Treat model-produced arguments as untrusted input: check types, ranges, and permissions before executing.
- **Gate side effects.** Reads can be automatic; writes, payments, and deletions should require confirmation or run in a sandbox.
- **Return useful errors.** A clear error string sent back lets the model correct itself on the next attempt.

MODULE 08 — KEY TAKEAWAYS

- The model proposes; your code disposes. Execution always stays on your side.
- Tool descriptions are part of your prompt surface — version and test them.
- Validate arguments and gate irreversible actions.



## LAB 08 — EXERCISES

The theme is the trust boundary: the model proposes, your code validates and disposes.

### E1 · Declare a tool [warm-up]

Write the JSON schema for `get_order_status(order_id)` backed by the repo's fake order table. Make the description say when *not* to use it.

### E2 · The two-round-trip loop [core]

Implement the full cycle: model requests the call, your code executes, result returns, model answers. Log each leg. Cap the number of rounds.

### E3 · Hostile arguments [core]

Have the model call your tool with an order ID that doesn't exist, then with a malformed one. Validate, return structured errors, and confirm the model recovers gracefully.

### E4 · Gate a dangerous tool [stretch]

Add `issue_refund` but require a typed confirmation before execution. The model may draft; only a human may fire.

SHIP IT — LAB CHALLENGE

Assemble a support bot with both tools: statuses answered automatically, refunds drafted but gated. Demonstrate one full conversation of each kind.

▸ Reference code: **labs/08-function-calling** in the companion repo. Try the exercises first, then compare.

# MCP

> *One open protocol between models and the tools and data they use.*

## THE INTEGRATION EXPLOSION

Function calling works, but every team ends up re-wiring the same integrations — one bespoke bridge per app-and-tool pair. The **Model Context Protocol** (MCP), introduced by Anthropic in 2024 and since adopted broadly across the industry, standardises that wire: any MCP *client* (a chat app, an IDE, an agent) can talk to any MCP *server* (GitHub, a database, your internal API) without custom glue. Build a server once; every MCP-capable host can use it.

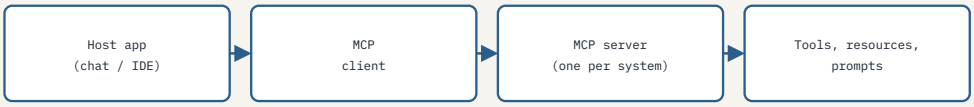



FIGURE 9.1 - ONE PROTOCOL BETWEEN ANY HOST AND ANY CAPABILITY

## WHAT A SERVER EXPOSES

- **Tools** — callable actions, exactly like Module 08's functions, discovered at runtime instead of hard-coded.
- **Resources** — readable data (files, records, documents) the host can pull into context.
- **Prompts** — reusable, parameterised prompt templates the server ships alongside its tools.

The client connects, asks the server what it offers, and presents those capabilities to the model — discovery replaces integration code.

## THE SECURITY TRADE

Plugging third-party capability into a model's context is powerful and risky in equal measure. A malicious or compromised server can attempt **prompt injection** through tool results; over-broad servers can exfiltrate data. Treat MCP servers like dependencies: pin versions, grant least privilege, keep dangerous operations behind human confirmation, and log every call.

MODULE 09 — KEY TAKEAWAYS

- MCP turns M×N custom integrations into M+N standard ones.
- Capabilities are discovered at runtime — hosts don't hard-code tools.
- Apply dependency-level scrutiny to any server you connect.



## LAB 09 — EXERCISES

One protocol, two sides: first consume it, then provide it.

### E1 · Speak the protocol [warm-up]

Run the repo's minimal MCP server under the MCP Inspector (or any MCP-capable host). List its tools and call one — note that you wrote zero integration code on the client side.

### E2 · Serve your own tools [core]

Wrap Lab 08's order tools in an MCP server. Connect from an MCP-capable host and use them from chat.

### E3 · Threat-model a server [stretch]

You're evaluating a third-party MCP server for production. Write the one-page review: what it can read, what it can do, how a poisoned tool result could inject instructions, and your three mitigations.

SHIP IT — LAB CHALLENGE

Publish your Lab 08 tools as an MCP server with a README covering scopes, dangerous operations, and logging — written for the security reviewer, not the user.

▸ Reference code: **labs/09-mcp** in the companion repo. Try the exercises first, then compare.

# AI Agents

> *Loops, not single calls: bounded autonomy toward a goal.*

## FROM ONE CALL TO A LOOP

Everything so far has been a single round trip. An **agent** wraps the model in a loop: it plans, takes an action with a tool, observes the result, and decides what to do next — repeating until the goal is met or a limit is hit. That loop is what lets a system handle tasks whose steps can't be enumerated in advance: “find the failing test, fix it, and open a PR” rather than “answer this question.”

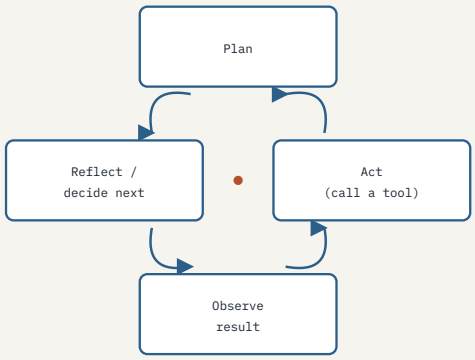



FIGURE 10.1 - THE AGENT LOOP; EXIT WHEN THE GOAL IS MET OR BUDGET IS SPENT

## DESIGNING THE LOOP

- **Bounded autonomy.** Cap iterations, tokens, wall-clock time, and spend. A stuck agent must fail loudly, not loop forever.
- **Small, sharp toolset.** Five well-described tools beat thirty vague ones; tool confusion is a top failure mode.
- **State outside the model.** Keep the task list, intermediate results, and history in your application; feed the model only what the next step needs.
- **Human gates.** Autonomy for reads and drafts; confirmation for anything irreversible.

Agents amplify both capability and error: a wrong turn early compounds across every later step. Start with the simplest workflow that could work — often a fixed pipeline with one decision point — and add autonomy only where it earns its keep.

MODULE 10 — KEY TAKEAWAYS

- Agent = model + tools + loop + stop conditions. The stop conditions are not optional.
- Reliability comes from constraints, not from a smarter model.
- Escalate to a human at exactly the actions you'd want a junior engineer to ask about.



## LAB 10 — EXERCISES

Every exercise here is about the constraints, not the intelligence — that's the module's thesis in practice.

### E1 · Bound the loop [warm-up]

Run the repo's agent with max-iterations, token budget, and wall-clock caps. Prove each cap triggers by setting it absurdly low.

### E2 · Fail loudly [core]

Give the agent an impossible task. Acceptable outcome: a clear failure report within budget. Unacceptable: silent looping. Fix the loop until the acceptable outcome is guaranteed.

### E3 · Add reflection [core]

Insert a “critique your last step before acting” stage. Measure the change in steps-to-completion across five tasks.

### E4 · Human gate [stretch]

Any file-writing action must print a preview and wait for y/n. Reads stay autonomous.

SHIP IT — LAB CHALLENGE

Build a file-organizer agent for a sandbox folder: it plans a re-organisation, shows the full move plan (dry-run), and executes only after approval — within budget, every time.

▸ Reference code: **labs/10-agents** in the companion repo. Try the exercises first, then compare.

# LangGraph

> *Agent workflows as explicit graphs: controllable, resumable, observable.*

## WHEN THE LOOP NEEDS STRUCTURE

A free-running agent loop is hard to debug, resume, or reason about. **LangGraph** models the workflow instead as an explicit *graph*: **nodes** are steps (call a model, run a tool, apply plain code), **edges** define what runs next, and **conditional edges** branch on the current state. A shared, typed **state object** flows through the graph and each node returns updates to it.

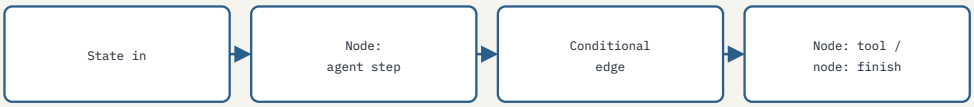



FIGURE 11.1 - EXPLICIT CONTROL FLOW: THE BRANCH DECISION IS CODE YOU CAN READ

## A GRAPH IN SIX LINES

```
from langgraph.graph import StateGraph, END

g = StateGraph(AgentState)
g.add_node("agent", call_model)      # decides: answer or use a tool
g.add_node("tools", run_tools)       # executes requested tool calls
g.add_conditional_edges("agent", should_continue,
                        {"tools": "tools", "end": END})
g.add_edge("tools", "agent")         # loop back after acting
app = g.compile()
```

## WHAT THE GRAPH BUYS YOU

- **Checkpointing** — state can persist after every node, so long tasks survive restarts and can resume mid-run.
- **Human-in-the-loop** — pause the graph at a node, wait for approval, continue: the Module 10 “human gate” as a first-class primitive.
- **Observability** — every node boundary is a natural place to log, trace, and test.
- **Multi-agent patterns** — supervisor/worker and other topologies become subgraphs rather than tangles of callbacks.

MODULE 11 — KEY TAKEAWAYS

- Graphs make agent control flow explicit, testable, and resumable.
- State is a typed object you design — the model only ever sees a rendered view of it.
- Reach for LangGraph when a plain loop stops being explainable.



## LAB 11 — EXERCISES

The deliverable is the same agent as Lab 10 — the point is what the graph makes visible, testable, and resumable.

### E1 · Loop → graph [warm-up]

Rebuild Lab 10's loop as a LangGraph `StateGraph` with agent and tool nodes. Confirm identical behaviour on the same tasks.

### E2 · Branch explicitly [core]

Add a conditional edge routing easy questions straight to answer and hard ones through tools. The routing function is plain Python — unit-test it.

### E3 · Kill it and resume [core]

Enable checkpointing, kill the process mid-run, restart, and watch it resume from the last node instead of the beginning.

### E4 · Interrupt for approval [stretch]

Replace Lab 10's y/n prompt with a graph interrupt before the tool node — approval becomes part of the graph itself.

SHIP IT — LAB CHALLENGE

A resumable research assistant: given a topic, it searches your Lab 07 index, drafts notes, pauses for review, and finishes after approval — surviving a restart at any point.

▸ Reference code: **labs/11-langgraph** in the companion repo. Try the exercises first, then compare.

# Evaluation

> *Numbers instead of vibes: measuring whether any of it actually works.*

## VIBES DON'T SCALE

Eyeballing a few outputs works for a demo and fails for a product. **Evaluation** is the practice of measuring quality systematically: a fixed dataset of inputs, a way to score outputs, and a habit of re-running the whole set on every meaningful change — new prompt, new model, new retrieval setting. Without it, you cannot tell an improvement from a regression.

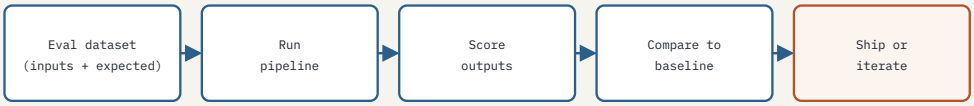



FIGURE 12.1 - THE EVAL LOOP: SAME DATASET, EVERY CHANGE, NO EXCEPTIONS

## THREE WAYS TO SCORE

- **Programmatic checks** — exact match, contains-string, JSON-schema validity, code that runs. Cheap, deterministic; use whenever the task allows.
- **LLM-as-judge** — a strong model grades outputs against a rubric (correctness, groundedness, tone). Scales to open-ended tasks; must itself be spot-checked against human judgment.
- **Human review** — the gold standard and the most expensive; spend it calibrating the other two and auditing high-stakes flows.

## BUILDING THE DATASET

Start small and real: 30–50 cases drawn from actual usage beat 500 synthetic ones. Include the ugly cases — ambiguous phrasing, missing context, adversarial input, questions whose correct answer is "I don't know." Every production failure you find gets added as a permanent regression test. For RAG, score retrieval separately from generation (did the right chunks come back?) — it tells you which half to fix.

MODULE 12 — KEY TAKEAWAYS

- An eval set is to an AI system what a test suite is to code.
- Layer scoring: programmatic where possible, LLM-judge where not, humans to calibrate.
- Grow the dataset from real failures — each one becomes a regression test.



## LAB 12 — EXERCISES

From here on, no prompt or pipeline change lands without this lab's script passing.

### E1 · Harvest real cases [warm-up]

Collect ten real inputs from your project's usage so far — include two ugly ones and one whose correct answer is "I don't know." Write expected outcomes for each.

### E2 · Programmatic scorers [core]

Write checkers for what can be checked mechanically: JSON validity, must-contain strings, correct refusals. Run the set and print a pass rate.

### E3 · LLM-as-judge, calibrated [core]

Write a grading rubric, have a strong model score each output 1–5, then score five yourself. If you and the judge disagree by more than a point on average, fix the rubric, not your opinion.

### E4 · Prove it catches regressions [stretch]

Deliberately break your prompt (delete the refusal instruction). The eval must visibly fail. If it doesn't, your dataset has a hole — add the missing case.

SHIP IT — LAB CHALLENGE

One command — `python run_eval.py` — that runs the dataset, prints pass rate and per-case results vs. the last baseline, and exits non-zero on regression. Wire it into the habit loop from Lab 01.

▸ Reference code: **labs/12-evaluation** in the companion repo. Try the exercises first, then compare.

# Production AI

> *Latency, cost, reliability, and safety once real users arrive.*

## CONTACT WITH REALITY

A pipeline that works in a notebook meets latency budgets, cost ceilings, hostile inputs, and provider outages the moment real users arrive. **Production AI** is the layer of engineering that absorbs those shocks without changing what the system does.




FIGURE 13.1 - A PRODUCTION REQUEST PATH; THE MODEL IS ONE HOP AMONG FIVE

## THE FOUR BUDGETS

- **Latency** — stream tokens to cut perceived wait; route simple requests to small fast models; parallelise retrieval and pre-work.
- **Cost** — cache aggressively (identical and semantically-similar requests), trim prompt bloat, and use the cheapest model that passes your evals per route.
- **Reliability** — retries with backoff, timeouts, circuit breakers, and a fallback model or degraded mode for provider outages.
- **Safety** — validate inputs against prompt injection, moderate outputs, scope every tool credential to least privilege, and log enough to reconstruct any incident.

## SHIPPING CHANGES SAFELY

Treat every prompt, model, and retrieval change like a deploy: run the Module 12 eval suite offline, then canary to a small slice of traffic, compare live metrics against the control, and keep one-click rollback. Model upgrades are not drop-in — behaviour shifts even between minor versions of the “same” model family.

MODULE 13 — KEY TAKEAWAYS

- Wrap the model: guards in front, checks behind, cache beside, fallback beneath.
- Route by difficulty — most traffic doesn't need your biggest model.
- Evals offline + canary online = the only safe way to change anything.

## LAB 13 — EXERCISES

Same bot as Lab 07 — now it can take a punch.

### E1 · Cache layer [warm-up]

Add exact-match response caching to your Lab 04 wrapper. Replay a day of test traffic and report the hit rate and cost saved.

### E2 · Survive an outage [core]

Simulate provider failure (the repo's `CHAOS=1` flag). Verify the fallback path engages, the user gets an answer, and the incident is logged.

### E3 · Guard the door [core]

Add an input check for prompt-injection patterns ahead of your RAG bot, and an output check that citations actually exist in the retrieved set.

### E4 · Canary a change [stretch]

Route 10% of eval traffic to a new prompt version. Compare pass rates and cost side by side before promoting to 100%.

SHIP IT — LAB CHALLENGE

Wrap your Lab 07 bot with all four budgets — guards in front, checks behind, cache beside, fallback beneath — and print a one-screen ops report: p50/p95 latency, cache hit rate, fallbacks.

▸ Reference code: **labs/13-production** in the companion repo. Try the exercises first, then compare.

# MLOps

> *Versioning, tracing, and monitoring that keep the system healthy over time.*

## KEEPING IT ALIVE

**MLOps** — and its LLM-era sibling often called *LLMOps* — is the operational discipline that keeps an AI system healthy over months, not demos. Classical MLOps centres on training pipelines and model registries; in an AI-engineering stack the same rigour applies to a different set of artifacts: **prompts, retrieval indexes, eval datasets, model versions, and configuration**.

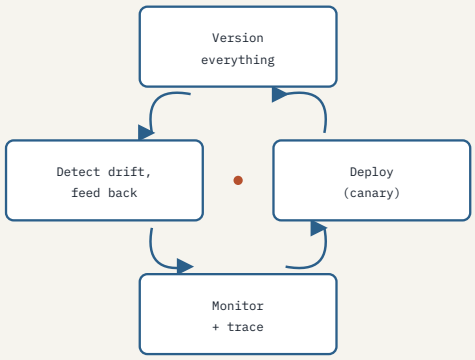



FIGURE 14.1 - THE OPERATIONS LOOP; MONITORING FEEDS THE NEXT ITERATION

## THE PRACTICES THAT MATTER

- **Version everything together.** A release is a pinned tuple: model ID + prompt version + index snapshot + config. Any answer in production must be reproducible from its tuple.
- **Trace end-to-end.** For each request, record the full chain — input, retrieved chunks, tool calls, output, latency, token cost — so any bad answer can be replayed and diagnosed.
- **Monitor quality, not just uptime.** Sample live traffic through your LLM-judge rubrics; watch refusal rates, groundedness, and user feedback alongside p95 latency and cost per request.
- **Watch for drift.** User behaviour shifts, documents go stale, providers silently update models. Scheduled eval runs against production snapshots catch decay before users do.

The loop closes where the course began: monitoring surfaces failures, failures become eval cases, eval cases drive the next prompt or pipeline change — and the build continues.

MODULE 14 — KEY TAKEAWAYS

- Reproducibility = versioned tuple of model, prompt, index, and config.
- Tracing turns “the bot said something weird” into a replayable bug report.
- Quality monitoring is continuous evaluation — the loop never finishes.

## LAB 14 — EXERCISES

The final lab turns your project from a demo into something you could hand to another engineer.

### E1 · Stamp the tuple [warm-up]

Attach a version tuple — model ID, prompt version, index content-hash, config hash — to every answer your bot produces. Any answer must be traceable to its exact build.

### E2 · Trace and replay [core]

Log every request end-to-end (input, retrieved chunks, output, latency) as JSONL. Add a replay command (the reference code uses `python trace.py <trace_id>`) that re-runs any past request against the current build and reports which part of the tuple changed.

### E3 · Schedule the evals [core]

Write the cron/CI job that runs Lab 12 nightly against the production tuple and alerts on regression. Dry-run it manually.

### E4 · Drift drill [stretch]

Swap in a different model version without telling your eval suite. Confirm the nightly run catches the behaviour change within one cycle.

SHIP IT — LAB CHALLENGE

Write the two documents every on-call engineer needs: a release checklist (what must pass before any tuple change ships) and a one-page incident runbook (bad answer reported → trace → replay → diagnose → add eval case). That closes the loop — and the course.

▸ Reference code: **labs/14-mlops** in the companion repo. Try the exercises first, then compare.

END OF BUILD

# You've Wired Up the Full Stack

*From tokens to production monitoring — the same pipeline, now assembled end to end.*

THE BUILD, COMPLETE



Next: pick one module and build the smallest working version of it.

14 / 14